

RIMBALZI DI UNA PALLINA

INTRODUZIONE

Nel modello più semplice che possiamo formulare una pallina elastica che rimbalza su una superficie rigida (e.g., il pavimento) perde una frazione γ della sua energia indipendente dalla velocità prima dell'impatto. Se lasciamo cadere la pallina da un'altezza nota h_0 , con velocità iniziale nulla, sotto l'azione della forza di gravità, l'altezza massima raggiunta dopo n rimbalzi sarà dunque, sotto queste ipotesi:

$$h_n = h_0 \gamma^n. \quad (1)$$

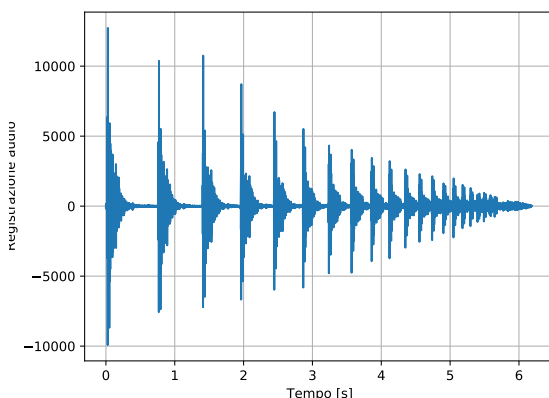
Lo scopo dell'esperienza è lo studio della dinamica dei rimbalzi di una pallina elastica utilizzando lo smartphone per registrare il rumore prodotto dai rimbalzi.

MATERIALE NECESSARIO

- Una pallina elastica.
- Metro a nastro.
- Smartphone o macchina fotografica digitale.

MISURE ED ANALISI

Misureremo i tempi dei vari rimbalzi attraverso l'analisi di una apposita registrazione audio eseguita con lo smartphone. Come potete vedere nel frammento di codice esemplificativo riportato nella pagina seguente, un file audio in formato *wave* può essere letto e trasformato in un grafico—a questo punto si può lavorare con lo strumento *zoom* di *matplotlib* ed il *mouse* per stimare i tempi.



ANALISI DEL FILE AUDIO

Analogamente a quanto fatto per l'esperienza della catenaria, misureremo i tempi di rimbalzo manualmente utilizzando il *mouse*. Notiamo che ogni rimbalzo è caratterizzato da un *attacco* estremamente

rapido e da una *coda* relativamente più lunga. Si consiglia di eseguire uno zoom stretto sui ciascun attacco per stimare con precisione il momento dell'impatto; dovreste anche, va da sé, pensare a come stimare in modo ragionevole l'incertezza associata.

(Man mano che la velocità di impatto sul suolo diminuisce, anche la pressione sonora si riduce, come si vede chiaramente dalla figura.)

STIMA DELL'ALTEZZA MASSIMA

È facile dimostrare che, nel nostro modello, l'altezza massima raggiunta tra due rimbalzi successivi può essere stimata a partire dai tempi di rimbalzo come

$$h_n = \frac{1}{8} g (t_n - t_{n-1})^2. \quad (2)$$

(Notiamo, per inciso, che questo non include un certo numero di effetti potenzialmente rilevanti come la resistenza dell'aria.)

Realizzate un grafico dell'altezza h_n in funzione dell'indice n del rimbalzo. Se il nostro modello descrive accuratamente i dati, questi ultimi dovrebbero disporsi come un esponenziale (e quindi dovrebbero apparire come una retta in scala semilogaritmica). Si fitti il grafico di dispersione con la (1) e si stimi il valore di *gamma*. Si confronti il valore di h_0 con la misura diretta dell'altezza iniziale.

CONSIDERAZIONI PRATICHE

Avrete bisogno di un ambiente il più possibile silenzioso per effettuare la registrazione. Palline diverse perderanno una frazione diversa della loro energia ad ogni rimbalzo, per cui si consiglia di fare un certo numero di prove per trovare la configurazione ottimale (inclusa la misura dell'altezza iniziale da cui la pallina viene fatta cadere).

Assicuratevi che la *app* che utilizzate permetta di registrare in formato *wave*—possibilmente mono. Ce ne sono svariate per tutte le piattaforme. Alternativamente avrete bisogno di un programma per convertire in formato *wave* il file prodotto dal vostro smartphone (e.g., in *mp3*). *Audacity*, reperibile da <https://www.audacityteam.org/>, è un esempio di editor audio open-source e multiplatforma, che vi permette di fare la conversione (oltre ad un numero sconfinato di altre cose).

```

1 import wave
2
3 import numpy as np
4 from matplotlib import pyplot as plt
5 from scipy.optimize import curve_fit
6
7 file_path = 'macro/data/bouncing_ball3.wav'
8 # Apertura del file audio, vedi https://docs.python.org/3/library/wave.html
9 stream = wave.open(file_path)
10 signal = np.frombuffer(stream.readframes(stream.getnframes()), dtype=np.int16)
11 # Importante: se il file originale e' stereo dobbiamo prendere solo uno dei canali.
12 if stream.getnchannels() == 2:
13     signal = signal[:,2]
14 # L'unica cosa che ci manca e' l'array dei tempi corrispondenti ai singoli campioni.
15 t = np.arange(len(signal)) / stream.getframerate()
16
17 # A questo punto siamo pronti per analizzare il tracciato audio. Una volta
18 # messi i dati in un grafico di matplotlib possiamo utilizzare lo strumento
19 # zoom ed il mouse per misurare i tempi di rimbalzo della pallina.
20 plt.figure('Rimbalzi pallina')
21 plt.plot(t, signal)
22 plt.xlabel('Tempo [s]')
23 plt.savefig('audio_rimbalzi.pdf')
24
25 # Supponiamo di aver preso a mano questa serie di tempi di rimbalzo.
26 t = [0.01961451, 0.69562358, 1.23981859, 1.679161, 2.04113379, 2.33979592]
27 # Cercate di stimare ragionevolmente gli errori...
28 sigma_t = 0.005
29 # Calcolo delle differenze di tempo.
30 dt = np.diff(t)
31 # Creazione dell'array con gli indici dei rimbalzi.
32 n = np.arange(len(dt)) + 1.
33 # Calcolo dell'altezza massima e propagazione degli errori.
34 h = 9.81 * (dt**2.) / 8.0
35 dh = 2.0 * np.sqrt(2.0) * h * sigma_t / dt
36
37 def expo(n, h0, gamma):
38     """Modello di fit.
39     """
40     return h0 * gamma**n
41
42 plt.figure('Altezza dei rimbalzi')
43 plt.errorbar(n, h, dh, fmt='o')
44 popt, pcov = curve_fit(expo, n, h, sigma=dh)
45 h0_hat, gamma_hat = popt
46 sigma_h0, sigma_gamma = np.sqrt(pcov.diagonal())
47 print(h0_hat, sigma_h0, gamma_hat, sigma_gamma)
48 x = np.linspace(0.0, 6.0, 100)
49 plt.plot(x, expo(x, h0_hat, gamma_hat))
50 plt.yscale('log')
51 plt.grid(which='both', ls='dashed', color='gray')
52 plt.xlabel('Rimbalzo')
53 plt.ylabel('Altezza massima [m]')
54 plt.savefig('altezza_rimbalzi.pdf')
55
56 plt.show()

```